Resolve now – project Document

1. Introduction

- **Project Title:** ResolveNow – Smart Complaint Management System
- **Team Members:**
  N.Mahalakshmi surya Nandini(Team leader)
  A.V.V.Bhargav
  M.Sri kanya
  M.Jhansi

2. Project Overview

- **Purpose:**
  ResolveNow is designed to simplify and streamline the process of registering, tracking, and resolving complaints. It empowers users (employees/customers) to submit complaints and allows administrators or support teams to manage and act on them efficiently.
- **Features:**
  - User Registration and Login (with authentication)
  - Submit New Complaint
  - View Complaint History
  - Admin Dashboard to View & Update Complaints
  - Email Notifications for Complaint Status Updates
  - Responsive UI with attractive design

3. Architecture

- **Frontend (React):**
  - Built using React and Vite
  - Component-based structure (Home, Login, Dashboard, ComplaintForm, etc.)
  - Axios for API requests
  - CSS modules & custom styles for UI
- **Backend (Node.js + Express):**
  - RESTful API built using Express.js
  - Handles routes for user auth, complaints, and admin operations
  - JWT-based Authentication Middleware
- **Database (MongoDB):**
  - Collections:
    - `users` – stores user info

        - `complaints` – stores complaint records
    - Mongoose for schema modeling and validation

4. Setup Instructions

- **Prerequisites:**
    - Node.js (v18+)
    - MongoDB (local or Atlas)
    - Git

- **Node.js & npm:** Required to run backend JavaScript.

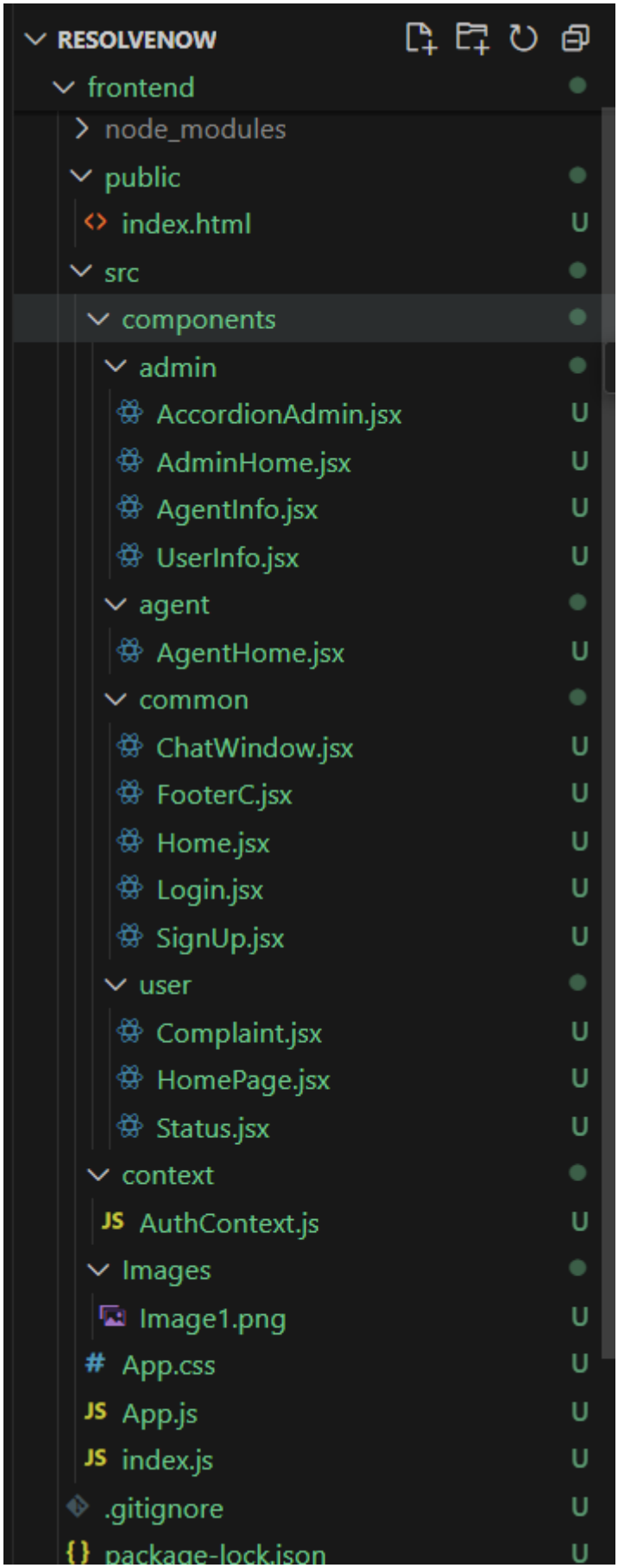
- **Express.js:** Web framework for building APIs. Install via `npm install express`
- **MongoDB:** NoSQL database.
- **React.js:** Frontend UI library.
- **HTML, CSS, JavaScript:** Essential web dev basics
- **Mongoose:** ODM to connect Node.js with MongoDB.
- **Optional UI Libraries:** Material UI, Bootstrap
- **Version Control:** Git for managing code.
- **Editor:** VS Code recommended

- **Installation:**

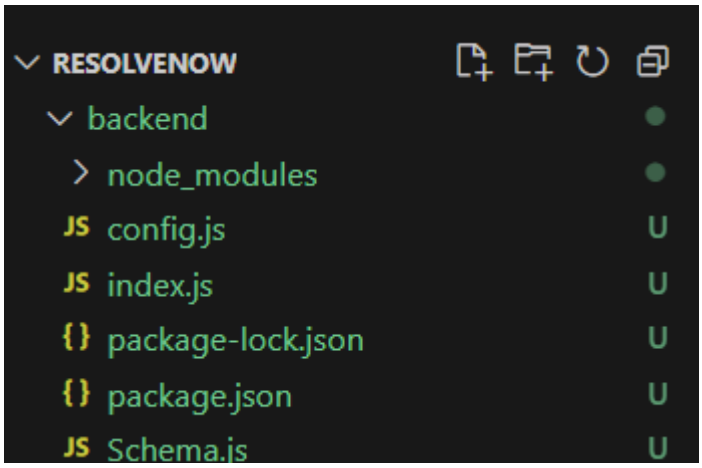
```
git clone
https://github.com/yourusername/resolvenow.git
cd resolvenow

# Install frontend
cd frontend
npm install

# Install backend
cd ../backend
npm install
```

5. Folder Structure

Client (`frontend/`):


RESOLVENOW
frontend
node_modules
public
index.html
src
components
admin
AccordionAdmin.jsx
AdminHome.jsx
AgentInfo.jsx
UserInfo.jsx
agent
AgentHome.jsx
common
ChatWindow.jsx
FooterC.jsx
Home.jsx
Login.jsx
SignUp.jsx
user
Complaint.jsx
HomePage.jsx
Status.jsx
context
AuthContext.js
Images
Image1.png
App.css
App.js
index.js
.gitignore
package-lock.json

Server (`backend/`)



6. Running the Application

- **Frontend:**

  ```
  cd frontend

  npm start
  ```

- **Backend:**

  ```
  cd backend

  node index.js
  ```

8. Authentication

- **JWT tokens stored in localStorage:** JSON Web Tokens are issued upon successful login and stored in the browser's localStorage. This allows persistent user sessions across page reloads. However, developers should be cautious of XSS attacks when using localStorage.
- **Protected routes on both client and server:** Frontend routes (e.g., accessing the dashboard or complaint page) are conditionally rendered based on token validity. On the backend, routes are protected by checking for valid tokens in the request headers before processing.

- **Middleware verifies token before granting access:** Middleware on the server intercepts incoming requests, extracts the JWT token, verifies it (e.g., using a secret key), and attaches the user info to the request object if valid. If invalid or missing, it blocks the request with a 401 Unauthorized response.

9. User Interface

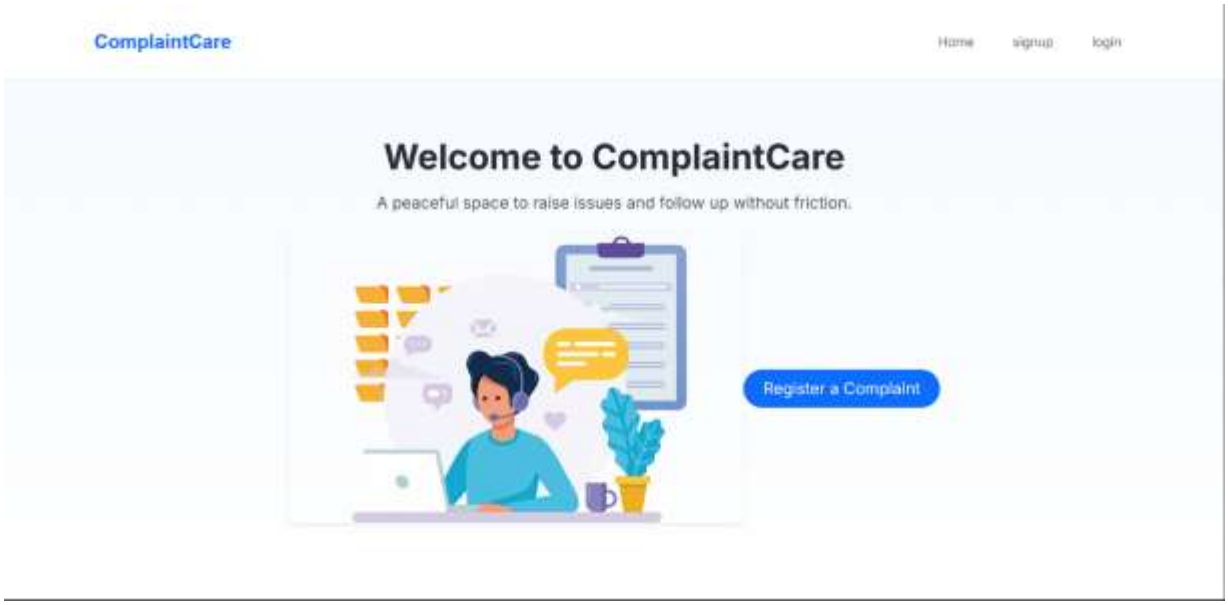
- **Clean and modern UI:** Designed with user experience in mind, often using frameworks like React, Material UI, or Tailwind CSS to ensure the layout is responsive, accessible, and visually appealing.
- **Home page with intro text and hero image:** Acts as a landing page introducing the purpose of the platform with compelling visuals and text that guide the user to either log in or sign up.
- **Login & registration forms:** Users can securely register and log in. Forms typically include input validation and user feedback for success or error states.
- **Complaint submission form:** Allows logged-in users to file a complaint, usually containing fields like subject, description, and category. It may include file uploads or screenshots as evidence.
- **Admin dashboard for complaint management:** Admin users can view, filter, and manage submitted complaints. Features often include status updates, assignment of complaints to departments, and communication with users.

10. Testing

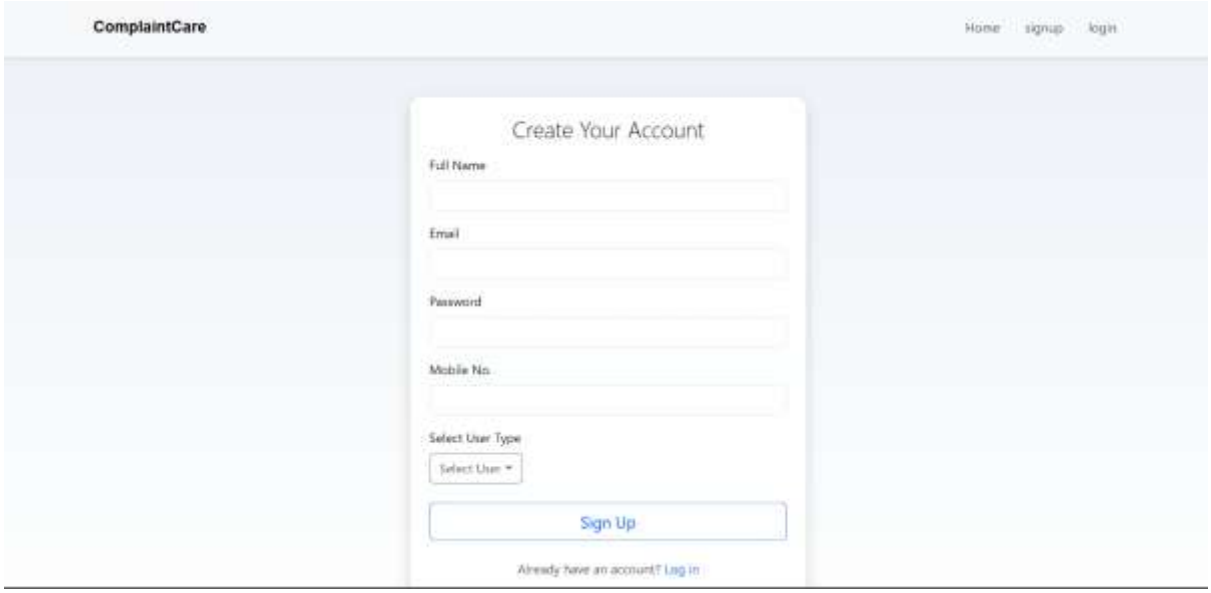
- **Manual testing via Postman:** Used to verify API endpoints—sending requests with different payloads and checking for correct status codes, error messages, and data responses.
- **Frontend tested in Chrome and Firefox:** Ensures consistent behavior and design across major browsers, catching UI glitches or compatibility issues.
- **API route validation and response checks:** Ensures that routes handle edge cases correctly (e.g., missing data, malformed input), and responses conform to expected schema and format.

11. Screenshots or Demo

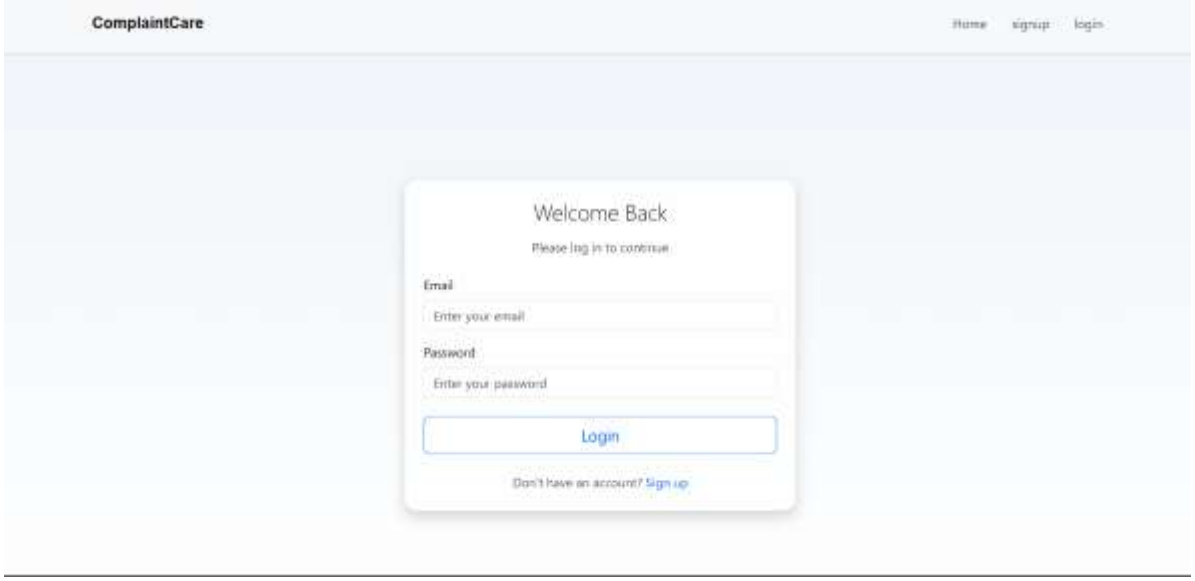
- Landing Page



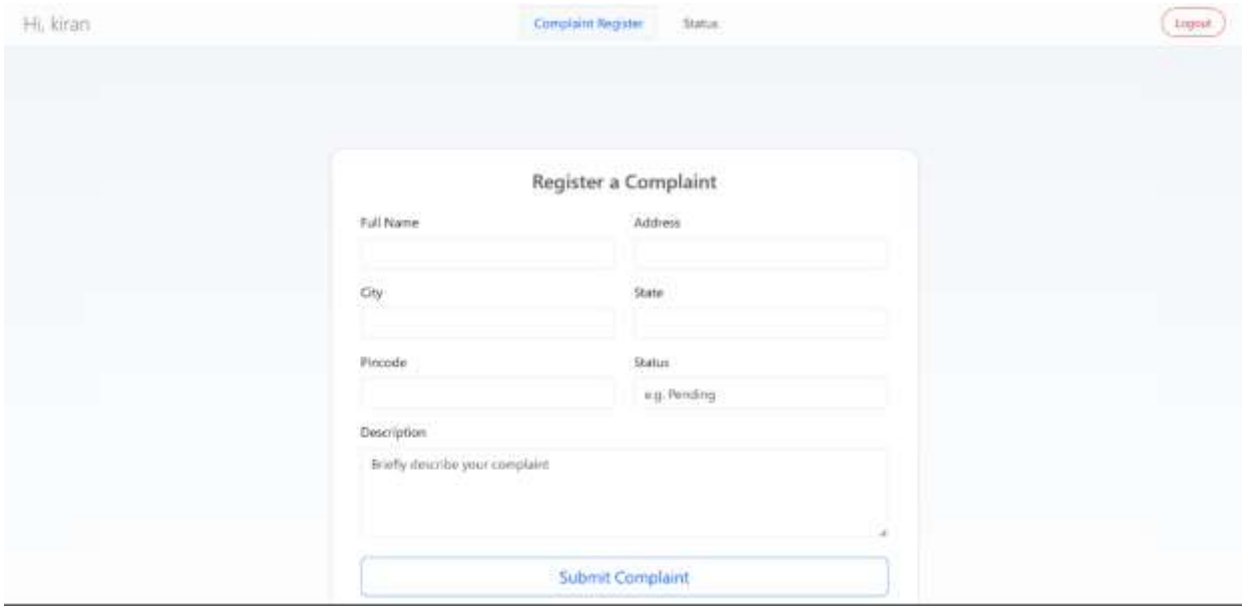
- Registration Page



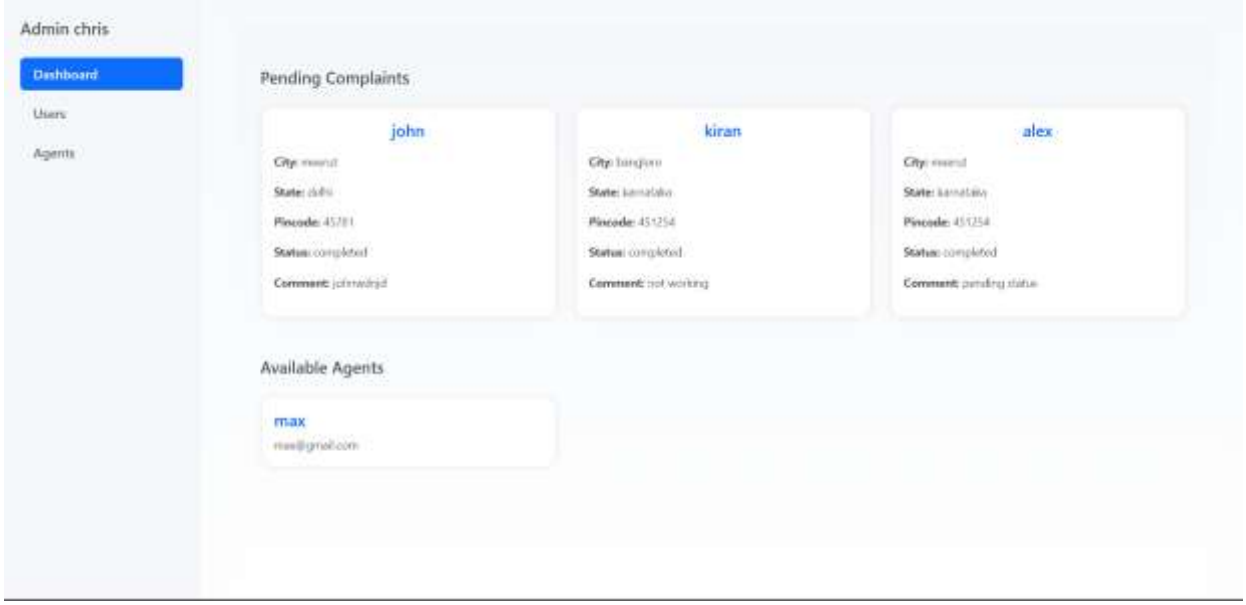
- Login Page



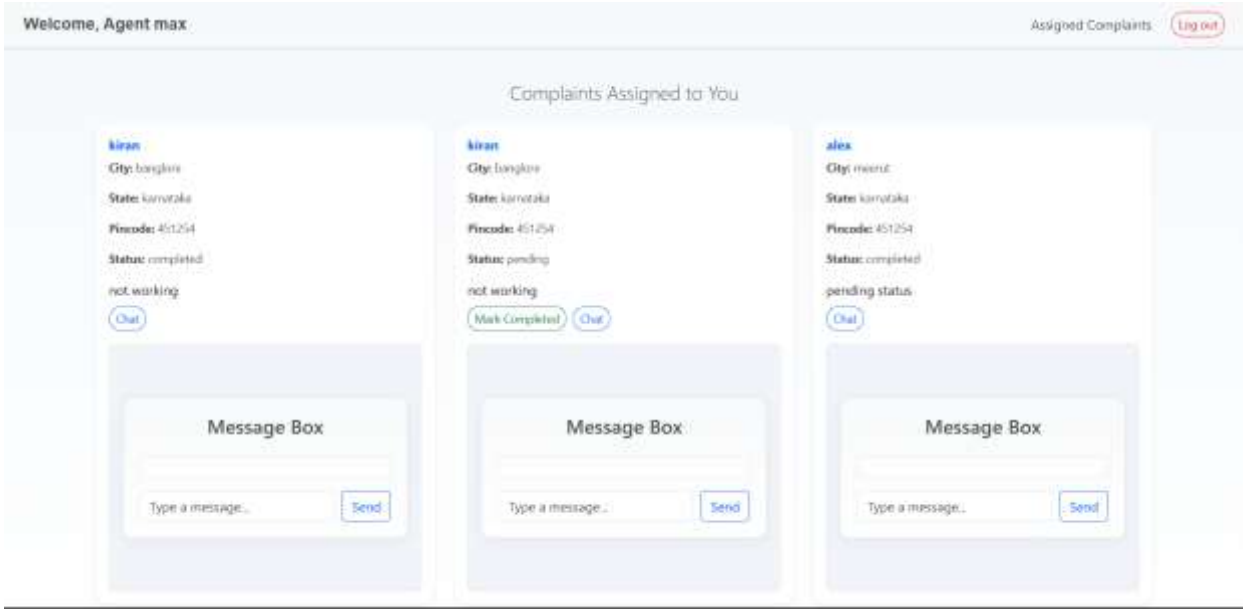
- Common Dashboard For Complaint



- Admin Dashboard



- Agent Dashboard



12. Known Issues

- **Admin panel not yet role-restricted (WIP):** Currently, there's no access control mechanism in place to prevent regular users from accessing admin-specific routes or components. This could lead to unauthorized visibility or manipulation of sensitive data. Implementing role-based middleware and conditional rendering on the frontend is underway to fix this.
- **No email notifications currently configured:** Users and admins currently receive no automated updates on complaint status changes, registration confirmations, or password resets. This affects user engagement and transparency, and could be improved with a backend email service integration (e.g. Nodemailer, SendGrid).

13. Future Enhancements

- **Email or SMS alerts for status updates:** The system will notify users when their complaints are acknowledged, resolved, or updated—either via email or SMS, depending on user preference. This improves responsiveness and trust in the platform.
- **Role-based access (Admin vs User):** Once implemented, each user will have clearly defined roles. Admins get access to complaint management tools, analytics, and moderation controls, while users interact solely with features relevant to them. This could be enforced using role claims within JWTs and conditional route logic both server- and client-side.
- **File upload with complaints:** Users will be able to attach evidence (photos, documents, etc.) when submitting complaints. This requires front-end support for file input, and secure backend handling using file storage systems like AWS S3 or local disk with validation to prevent malicious uploads.
- **Search & filter functionality for admins:** As complaints grow, so does the need for streamlined navigation. Search by keywords, filter by status, category, date range, or user—these tools will help admins prioritize, organize, and manage submissions more efficiently